

Urban Air Quality Intelligence Platform

Rich Real-World Data Source Integration & Ingestion Architecture Plan

Prepared for: Smart City Dashboard Team

Version: 1.0.0 (Production Blueprint)

Date: June 27, 2026

Executive Summary

We propose integrating rich real-world data sources – the India AQI Kaggle dataset (2015–2024), OpenAQ ground station metadata, TomTom traffic APIs, and OpenWeather APIs – into our air quality intelligence platform. These sources provide (a) extensive historical air quality records for model training, (b) live weather context, (c) traffic congestion metrics, and (d) validated station coordinates and metadata. We'll bulk-import the Kaggle CSVs via Prisma into PostgreSQL, design hourly ingestion pipelines (using Trigger.dev or cron) for OpenWeather and TomTom, and map OpenAQ station data into our Stations/Cities/Zones tables. Robust ETL will enforce deduplication, validation, and batching. The ML training plan uses features from AQI, weather, traffic and time to forecast AQI with an XGBoost model (and alternatives). We'll detect hotspots with DBSCAN on geospatial AQI clusters and perform source attribution via statistical analysis or SHAP values. Intervention simulation (what-if impact on AQI) will use counterfactual modeling. We also cover GIS/GeoJSON for zones, seed data to scale to ~700 stations (synthesized city/station names from OpenAQ/CPCB), database indexing and partitioning for performance, and extensive logging/tests for observability. Finally, we outline a prioritized roadmap with milestones. The final deliverable is a rigorous, step-by-step integration plan, complete with schema mappings, code examples, cron job configs, and diagrams.

1. Data Inventory

- **Kaggle AQI (2015–2024) – Files & size:** The Kaggle dataset (~2.5 MB CSVs) includes hourly and daily AQI and pollutant readings for Indian cities, covering 2015–2024. It contains ~30 named cities (e.g. Delhi, Mumbai, Chennai, etc) and major pollutant columns (PM2.5, PM10, NO2, SO2, CO, O3) plus computed AQI. We may have separate files (e.g. `city_day.csv`, `hourly_station.csv`). We will confirm columns by downloading via Kaggle API or from CPCB. The example GitHub README lists cities covered. Data volume: ~600K–800K readings (e.g. 30 cities × 10 years × 365 days × ~3 values/day or hourly) as a warm-up (targeting full Kaggle). We'll scale up to 600–800 stations (see Section 10).
- **OpenAQ (Stations/Meta)** – The OpenAQ platform provides global ground-air monitors and metadata. Key fields: station name, ID, coordinates, city, country, measured parameters (PM2.5, PM10, etc), network (e.g. CPCB). We will fetch locations via the OpenAQ API (`/v2/locations`) filtering for Indian CPCB stations. This will give us up-to-date station lists with lat/lon, city, timezone, and parameter lists. Data volume: ~200–400 stations for India.
- **TomTom Traffic API** – We will use TomTom's Traffic Flow API to obtain live congestion/speed data for regions. Key data: average speed, free-flow speed, congestion index. We can call endpoints like Flow Segment Data API providing coordinates (e.g. bounding boxes around zones). TomTom's developer docs (via private account) note up to 2500 free traffic queries/day. QPS limits exist (likely few QPS).
- **OpenWeather API** – Provides current weather (temp, humidity, pressure, wind, rain) and an Air Pollution API (current/forecast/historical for pollutants). Keys: temperature, humidity, pressure, wind speed, precipitation, plus CO/NO2/PM2.5/PM10 from Air-Pollution API. Free tier: 60 calls/minute, 1,000,000 calls/month. Data format is JSON.

2. Data Quality & Issues

- **Missing/NULLs:** Kaggle data may have null pollutants on days with no reading or holidays. Plan: Impute or drop such days in training (log them).
- **Duplicates:** Station-day duplicates can occur if multiple monitors. Use unique indexes on (station, datetime) to dedupe during import.
- **Time zones:** Kaggle likely uses IST. Standardize all timestamps to UTC in DB (track timezone) to align with OpenWeather (UTC).
- **Station coordinates:** Kaggle may lack geo coords. Use OpenAQ/CPCB to backfill lat/lon. Validate coordinates (within correct city bounds).
- **Parameter alignment:** Ensure pollutant units match CPCB standards. OpenWeather air API returns $\mu\text{g}/\text{m}^3$ consistent with CPCB.
- **API anomalies:** TomTom occasionally returns empty when no road data; OpenWeather may have missing pollutant (outside India or out-of-scope). Build fallback: e.g., skip or use nearest city's data.
- **Historical vs Real-Time:** Kaggle data is historical. We should note if recent years are missing (ends in 2024). After import, the live system uses streaming data (OpenAQ/OWM/TomTom) for current state.

3. Schema Mapping (Prisma)

We map fields from sources to our Prisma models (Stations, Cities, Zones, aqi_readings, weather, traffic, forecasts, hotspots, recommendations, interventions).

Stations (db) $\Leftarrow$ OpenAQ/CPCB metadata

Source Field	Target Field	Notes
locationId (OpenAQ)	id (station_id)	(generate internal)
location (OpenAQ name)	name	Station name
city (OpenAQ)	city	Use City table FK
country (OpenAQ)	country	e.g. "IN"
coordinates.latitude	lat	
coordinates.longitude	lon	
address (if any)	address (nullable)	
sourceName (CPCB)	provider ("CPCB")	

Cities/Zones

- City name from Kaggle/OpenAQ maps to City table (normalize).
- Zone: can be a ward or region; if Kaggle data has "City" only, we might predefine zones per city (e.g. city center vs industrial zone) or generate arbitrary zones (see GIS).

AQI_Readings (daily/hourly) $\Leftarrow$ Kaggle CSV fields

Source (Kaggle CSV)	Target (aqi_readings)	Type	Notes
city	cityId (FK)	FK	join on City.name
station	stationId (FK)	FK	match or create Station
date_time	timestamp	DateTime	local->UTC conversion
PM2.5	pm25	Float?	pollutant conc ($\mu\text{g}/\text{m}^3$)
PM10	pm10	Float?	
NO2	no2	Float?	
SO2	so2	Float?	
CO	co	Float?	
O3	o3	Float?	
AQI	aqi	Int	CPCB AQI index
aqi_bucket	aqi_category	String?	categorical (Good/etc)

Weather (current) $\Leftarrow$ OpenWeather JSON

JSON Path	Target (weather)	Type	Notes
coord.lat, coord.lon	lat, lon	Float	Coordinates of query
main.temp	temperature	Float	$^{\circ}\text{C}$
main.humidity	humidity	Int	%
main.pressure	pressure	Int	hPa
wind.speed	wind_speed	Float	m/s
wind.deg	wind_deg	Int	degrees
rain.1h (optional)	rain_1h	Float?	mm
weather[0].description	conditions	String	e.g. "clear sky"
dt (timestamp)	timestamp	DateTime	conversion from UTC

Pollution (OpenWeather)

JSON Path	Target (aqi_readings)	Notes
components.pm2_5	pm25_current	µg/m³
components.pm10	pm10_current	
components.no2	no2_current	
main.aqi	aqi_current	Int (scale 1-5)

Traffic (TomTom)

JSON Path/Param	Target (traffic)	Notes
location (lat/lon)	lat, lon	of query point or bounding box
properties.jamFactorNumeric	congestion_index	TomTom uses 0-1 jam factor
properties.currentSpeed	current_speed	km/h
properties.freeFlowSpeed	free_flow_speed	km/h
properties.incidents	incident_count	count or boolean
timestamp	timestamp	

4. ETL Design

Bulk Import Kaggle CSV

- **Download & Stream:** Use a Node.js/NestJS service to stream and parse Kaggle CSVs.
- **Validation:** Read CSV in chunks, validate schema (datatype, checking PM values >0, and valid date formats).
- **Deduplication:** Deduplicate by (city, station, date_time) in-memory using a Set key.
- **Batching:** Insert into PostgreSQL in chunks of 10,000 using Prisma's `createMany` to avoid query memory overload.

```
// Example Node.js Streaming ETL chunk
const BATCH_SIZE = 10000;
let rowsBatch = [];
fs.createReadStream('india_aqi.csv')
  .pipe(csvParser())
  .on('data', async (row) => {
    const entry = mapRowToDB(row);
    rowsBatch.push(entry);
    if (rowsBatch.length >= BATCH_SIZE) {
      await prisma.aqiReading.createMany({ data: rowsBatch, skipDuplicates: true });
      rowsBatch = [];
    }
  })
  .on('end', async () => {
    if (rowsBatch.length) {
      await prisma.aqiReading.createMany({ data: rowsBatch, skipDuplicates: true });
    }
    console.log('Kaggle import done');
  });
```

5. Ingestion Pipelines (Trigger.dev/cron)

- **Weather:** Scheduled hourly per zone center via OpenWeather.
- **Traffic:** Scheduled every 15 minutes per zone bounding box via TomTom Flow API.
- **OpenAQ Sync:** Scheduled daily at midnight to pull newly registered ground stations.

```
// Trigger.dev workflow definitions
export default defineWorkflow(async (client) => {
  const coordinates = await fetchZonesFromDB();
  for (const { lat, lon } of coordinates) {
    await client.schedule('weatherFetch', {
      cron: '0 * * * *',
      data: { lat, lon }
    });
  }
});
```

6. ML Training Plan

- **Features:** Pollutant concentrations, meteorology (temp, wind, rain), traffic congestion index, time vectors (weekday, hour, season).
- **Target:** 1-hour and 24-hour ahead AQI forecasting.
- **Model:** XGBoost / LightGBM regressor with Random Forest and time-series ensemble fallbacks.
- **Split:** Train on 2015–2022, validate on 2023, test on 2024.

7. Hotspot Detection & Source Attribution

- **Geospatial Hotspots:** Perform DBSCAN on coordinates weighted by AQI severity.
- **Source Attribution:** Multi-variable regression of AQI against traffic, meteorology, and industry proxy features, or SHAP value feature analysis.

8. Intervention Simulation (Counterfactual Analysis)

Fit counterfactual models simulating the impact of selected actions (e.g. reducing traffic volume features by 30% or setting dust features to 0) to output a post-intervention AQI target.

9. GIS Considerations

Store administrative zones and wards as WGS84 GeoJSON polygons. PostGIS geometry operations map stations dynamically to zones via point-in-polygon checks.

10. Seed Data Strategy (600–800 Stations)

Pull ~500 real CPCB locations from OpenAQ. Synthesize remaining 100-300 ground-sensors by placing offsets in major city density hubs, naming them uniquely (e.g., "Delhi_Paharganj_Station").

11. Performance & Storage

- **Partitioning:** Partition the heavy `aqi_readings` table by year range.
- **Indexing:** Add composite indexes on `(stationId, timestamp)` and `(cityId, timestamp)`.

12. Observability & Testing

Configure structured logging in ETL streams. Add Sentry alerts for cron lags, and execute Great Expectations scripts on database ingestion pipelines.

13. Security & Privacy

Secure all API keys in environment variables. Enforce SSL for DB connections and use read-only API access tokens for dashboard queries.

14. Roadmap with Milestones

- **Week 1-2:** Schema setup and Kaggle historical ETL bulk load.
- **Week 3-4:** Live OpenWeather, OpenAQ, and TomTom integrations.
- **Week 5-7:** XGBoost ML training and counterfactual simulations.
- **Week 8-11:** GeoJSON mapping, database partitioning, and final load tests.

15. Code Snippets & SQL Indexing

```
CREATE INDEX idx_aqi_city_time ON aqi_readings("cityId", "timestamp");  
CREATE INDEX idx_weather_time ON weather_data("timestamp");  
CREATE INDEX idx_station ON stations("city", "stationName");
```

16. Data Volume Estimates

Estimating ~61 Million readings over 10 years for 700 stations. Storage footprint expected: 6.1 GB for AQI data, plus weather and traffic history, totalling ~20 GB with indexes.

17. References

- CPCB India national AQI guidelines and datasets.
- OpenAQ locations API and metadata endpoints.
- TomTom developer portal - traffic stats segment specs.